Strings

Session Objectives

- Understand string indexing and slicing
- Explore common string methods and operations

Deep Copy Vs Shallow Copy

id → address of a variable in Memory

Deep Copy

```
a = [1,2,3,4,5,6,7]
b = a # Deep Copy
b.append(9)
print(a == b) True
print(a is b) True
```

Shallow Copy

```
a = [1,2,3,4,5,6,7]
b = a.copy() # Shallow Copy
b.append(9)
print(a == b) False
print(a is b) False
```

→ independent table

Memory Diagram

a = [1,2,3,4,5,6,7,9]

b

Memory Diagram

a = [1,2,3,4,5,6,7]

b = [1,2,3,4,5,6,7,9]

b = a creates a variable b and points to same memory loacation of a
b = a.copy() creates a variable and creates a different memory location with same data as a.

```python
# Deep Copy
a = [1,2,3,4,5,6,7]
b = a # Deep Copy
b.append(9)
print(a == b)
print(a is b)
```

```
True
True
```

```python
# Address of the Variable a & b
# id Function
print(id(a))
print(id(b))
```

```
2203652252928
2203652252928
```

```python
a = [1,2,3,4,5,6,7]
b = a.copy() # Shallow Copy
b.append(9)
print(a == b)
print(a is b)
```

```
False
False
```

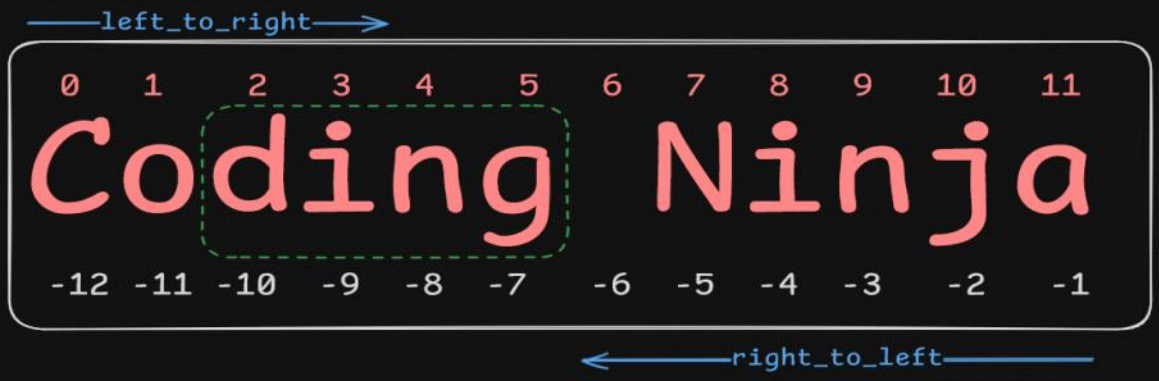
```python
# Shallow Copy will create an independent addresses
print(id(a))
print(id(b))
```

```
2203692749888
2203692750528
```

Slicing And Indexing

Indexing is allowing you to extract the char by passing index.
slicing is allowed to extract a sub-string from a string

[] -> Bracket Notation




```python
# Strings : Indexing & Slicing
# Python is a Programming Language having 'zero' based indexing
_str = "Coding Ninjas"
print(_str[0]) # 'C'
print(_str[-1]) # 's'
print(_str[-6]) # 'N'
print(_str[6]) # ' '
print(_str[-9]) # 'n'
print(_str[2]) # 'd'
```

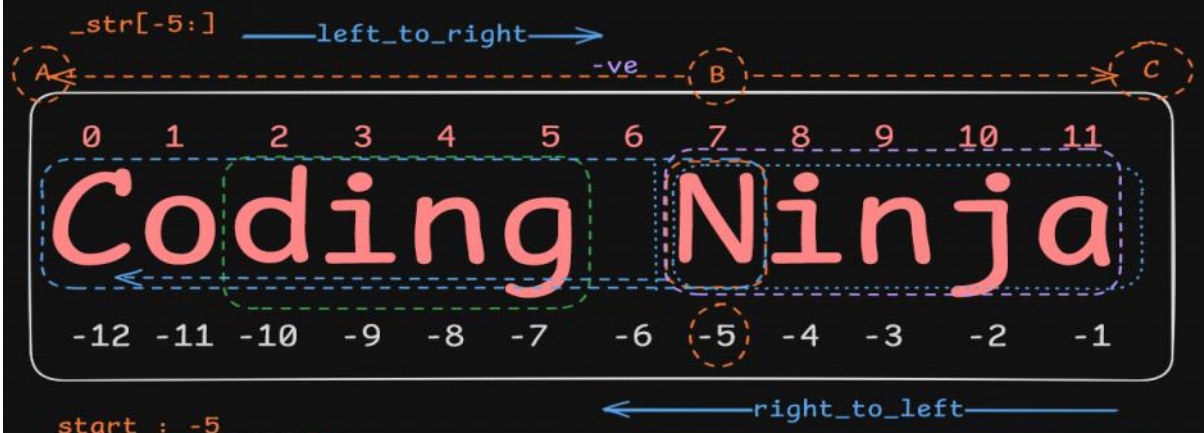
```
C
s
N
 
n
d
```

Slicing[Start : Stop : Step]

Start: 0 by default

Stop: length by default : Non-Inclusive

Step: A jump from one char to another
By default +1



start : -5
Stop : [Len , -1]
Step : +1 [direction , magnitude]

```python
print(_str[-5:12:-1])
```

direction: +ve [Forward]
direction: -ve [Backward]

```python
print(_str[-5::-1])
```

```
print(_str[15]) # IndexError: string index out of range
```

```
_str = "Coding Ninjas"
_str_length = len(_str)
print(_str_length) # 13
```

```
13
```

```
# Slicing [Start(0) : Stop(len) : Step(+1)]
# Stop [Non-Inclusive]
_str = "Coding Ninjas"
print(_str[:6]) # "Coding" # [0:6:1]
print(_str[7:]) # "Ninjas" # [7:13:1]
print(_str[0:12:2]) # 'Cdn ij'
print(_str[0:12:3]) # 'Ci n'
print(_str[:]) # "Coding Ninjas"
```

```
Coding
Ninjas
Cdn ij
Ci n
Coding Ninjas
```

```
print(_str[-5:]) # 'injas'
```

```
injas
```

```
# Stop [Non-Inclusive]
_str = "Coding Ninja"
print(_str[-5:]) # "Ninja"
```

```
Ninja
```

```
print(_str[-5::-1]) # "N gnidoC"
```

```
N gnidoC
```

```
print(_str[-5:12:-1]) # ""
```

```
print(_str[-5:12:+1]) # "Ninja"
```

```
Ninja
```

```
print(_str[-5:100:+1]) # "Ninja"
```

```
Ninja
```

```
print(_str[-5:100:0]) # ValueError: slice step cannot be zero
```

```
print(_str[-5::-2]) # Path is B to A [-ve direction] 2 is magnitude
```

```
Ngio
```

```python
_str = "Coding Ninja"
print(_str[-3:0:-1]) # 'niN gnido'
print(_str[-3::-1]) # 'niN gnidoC'
```

```
niN gnido
niN gnidoC
```

```python
_str = "Coding Ninja"
print(_str[-9:-2:-1]) # ""
print(_str[-9:-2:]) # "ing Nin"
```

```
ing Nin
```

```python
# String Cases Methods
_str = "Python is Awesome 🔥"
print(len(_str))
```

```
19
```

```python
print(_str.upper())
```

```
PYTHON IS AWESOME 🔥
```

```python
print(_str.lower())
```

```
python is awesome 🔥
```

```python
print(_str.title())
```

```
Python Is Awesome 🔥
```

```python
print(_str.capitalize())
```

```
Python is awesome 🔥
```

```python
# String Operations
str1 = 'Coding'
str2 = 'Ninjas'
# 'N' > 'C' : True
print(str2 > str1) # Boolean Return [Comparison]
```

```
True
```

```python
str1 = 'Coding'
str2 = 'Ninjas'
new_string = str1 + " " + str2 # Concatenation
print(new_string)
```

```
Coding Ninjas
```

```python
name = "Paras Chand"
age = 29
print(f"Hi , {name}. You are {age} year old.")
```

```
Hi , Paras Chand. You are 29 year old.
```

```python
# TypeError: can only concatenate str (not "int") to str
name = "Paras Chand" # 'str'
age = 29 # 'int'
print("Hi " + name + ",You are " + str(age) + " year old.")
```

```
Hi Paras Chand,You are 29 year old.
```

```python
print(type(age)) # 'int'
```

```
<class 'int'>
```

```python
# Repeat (*)
_echos = "Ninjas\n"
print(_echos * 5)
```

```
Ninjas
Ninjas
Ninjas
Ninjas
Ninjas
```

97 > 65

'a' > 'A'

'a'-'z' > 'A'-'Z'

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

www.alpharithms.com

"coding" > "Coding" : True

C[99] > C[67]

"momentum" > "moMentumABCD"

'm' > 'M'

```python
# String Comparison
_str1 = "Python"
_str2 = "Java"
print(_str1 == _str2) # False
print(_str1 != _str2) # True
print(_str1 >= _str2) # True
print(_str1 <= _str2) # False
```

```
False
True
True
False
```

```python
# String Comparison
_str1 = "Swim"
_str2 = "Swimming"
print(_str1 == _str2) # False
print(_str1 != _str2) # True
print(_str1 >= _str2) # False
print(_str1 <= _str2) # True
```

```
False
True
False
True
```

```python
# String Comparison
_str1 = "coding"
_str2 = "Coding"
# 'c' > 'C'
print(_str1 == _str2) # False
print(_str1 != _str2) # True
print(_str1 >= _str2) # True
print(_str1 <= _str2) # False
```

```
False
True
True
False
```

```python
# String Comparison
_str1 = "coding"
_str2 = "Zodiac"
# 'c'[99] > 'Z'[90]
print(_str1 == _str2) # False
print(_str1 != _str2) # True
print(_str1 >= _str2) # True
print(_str1 <= _str2) # False
```

```
False
True
True
False
```

```python
# String Comparison
_str1 = "codING"
_str2 = "coDiNg1234"
# 'd' > 'D'
print(_str1 == _str2) # False
print(_str1 != _str2) # True
print(_str1 >= _str2) # True
print(_str1 <= _str2) # False
```

```
False
True
True
False
```

```python
# Common String Methods....
# replace
_str = "Python is Awesome!"
new_str = _str.replace('Awesome','Fantastic')
print(new_str)
```

```
Python is Fantastic!
```

```python
# replace
_str = "Python is Awesome!"
new_str = _str.replace('o','0')
print(new_str)
```

```
Pyth0n is Awes0me!
```

```python
# Common String Methods....
# replace
_str = "Python is Awesome!"  arguments
new_str = _str.replace('Awesome','Fantastic')
print(new_str)
```

```
Signature: _str.replace(old, new, count=-1, /)
Docstring:
```

```
Python is Fantastic
```

Parameters

```python
# replace
_str = "Python is Awesome!"
new_str = _str.replace('o','0',1)
print(new_str)
```

```
Pyth0n is Awesome!
```

```
INSERT INTO TABLE_NAME(col1,col2,col3) # Parameter
VALUES('val1','val2','val3') # Arguments
Put arguments corresponding to the Parameters....
```

```python
# split()
# 'delimiter' -> '@'
_text = "Gurugram@Haryana@India@Asia"
print(_text.split("@"))
```

```
['Gurugram', 'Haryana', 'India', 'Asia']
```

```python
topics = "Python#Numpy#Pandas#Matplotlib#Seaborn#SQLAlchemy#MachineLearning#Statistics"
topics = topics.split("#")
print(topics)
print(type(topics)) # 'List'
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn', 'SQLAlchemy', 'MachineLearning', 'Statistics']
<class 'list'>
```

```python
print(topics[6]) # MachineLearning
print(topics[-1]) # Statistics
```

```
MachineLearning
Statistics
```

```python
topics = "Python#Numpy#Pandas#Matplotlib#Seaborn@SQLAlchemy@MachineLearning@Statistics"
# topics = topics.split("#").split("@") # AttributeError: 'list' object has no attribute 'split'
topics = topics.split("#")
print(topics)
print(type(topics)) # 'List'
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn@SQLAlchemy@MachineLearning@Statistics']
<class 'list'>
```

```python
topics[-1]
```

```
'Seaborn@SQLAlchemy@MachineLearning@Statistics'
```

```python
type(topics[-1])
```

```
str
```

```python
extended_list = topics[-1].split('@')
print(extended_list)
```

```
['Seaborn', 'SQLAlchemy', 'MachineLearning', 'Statistics']
```

```python
for topic in extended_list:
    topics.append(topic)

print(topics)
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn@SQLAlchemy@MachineLearning@Statistics', 'Seaborn', 'SQLAl
chemy', 'MachineLearning', 'Statistics']
```

```python
del topics[4]
print(topics)
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn', 'SQLAlchemy', 'MachineLearning', 'Statistics']
```

```python
topics = "Python#Numpy#Pandas#Matplotlib#Seaborn@SQLAlchemy@MachineLearning@Statistics"
# topics = topics.split("#").split("@") # AttributeError: 'list' object has no attribute 'split'
topics = topics.split("#")
print(topics)
print(type(topics)) # 'List'
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn@SQLAlchemy@MachineLearning@Statistics']
<class 'list'>
```

```python
extended_list = topics[-1].split('@')
print(extended_list)
```

```
['Seaborn', 'SQLAlchemy', 'MachineLearning', 'Statistics']
```

```python
topics.extend(extended_list)
print(topics)
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn@SQLAlchemy@MachineLearning@Statistics', 'Seaborn', 'SQLAl
chemy', 'MachineLearning', 'Statistics']
```

```python
topics.pop(4)
print(topics)
```

```
['Python', 'Numpy', 'Pandas', 'Matplotlib', 'Seaborn', 'SQLAlchemy', 'MachineLearning', 'Statistics']
```

```python
file_path = "http://localhost:8888/notebooks/anaconda_projects/CN-05/PythonLearning.ipynb?"
path_list = file_path.split("/")
path_list
```

```
['http:',
 '',
 'localhost:8888',
 'notebooks',
 'anaconda_projects',
 'CN-05',
 'PythonLearning.ipynb?']
```

```python
print(path_list)
```

```
['http:', '', 'localhost:8888', 'notebooks', 'anaconda_projects', 'CN-05', 'PythonLearning.ipynb?']
```